Vladmir installation and usage guide

Scope: source on branch main, inspected on October 14, 2026. The instructio
ns below do not claim that the software was installed or run in this enviro
nment.

1. What Vladmir is

Vladmir is a local Linux assistant with a desktop application, Ollama-serve
d models, memory and agenda stored locally, and optional integrations. Sepa
rate code exists for Android, Companion, and Wear OS. Many features require
 external programs, permissions, models, and configuration. Installation do
es not include AI weights, credentials, a ready APK, or a configured remote
 server.

2. Basic requirements

- Linux and Python 3.11 or later.
- Ollama installed and running at http://127.0.0.1:11434.
- A chat model and, optionally, a programming model already downloaded in O
llama.
- PyQt6 and dependencies from requirements.txt for the desktop UI.
- Space for the Python environment, models, and data. Model storage depends
 on the selected models.

Optional features have separate dependencies: ffmpeg and PulseAudio/PipeWir
e for audio and camera; Whisper and model files for transcription; Piper an
d an installed voice for synthesis; ClamAV for scans; gh for GitHub authent
ication; Android Studio/SDK/JDK to build apps; and a local MCP server enabl
ed inside a JetBrains IDE.

3. Install the desktop app

The documented procedure installs the checkout and a virtual environment in
 the user’s home directory:

‘‘bash
mkdir -p ~/.local/share/vladmir
git clone https://github.com/thu12dta3/Vladmir.git ~/.local/share/vladmir/s
ource
cd ~/.local/share/vladmir/source
python -m venv ~/.local/share/vladmir/core-env
~/.local/share/vladmir/core-env/bin/pip install -r requirements.txt
install -Dm755 scripts/vladmir ~/.local/bin/vladmir
install -Dm755 scripts/vladmir-desktop ~/.local/bin/vladmir-desktop
‘

The root README also includes commands to install the icon and application-
menu shortcut. If ~/.local/bin is not in PATH, launch by full path or adjus
t PATH for the user session.

Before launching, inspect project checks and options:

‘bash
cd ~/.local/share/vladmir/source
./scripts/vladmir --check
‘

This check inspects dependencies, configuration, and model availability. It
 does not prove that a disconnected camera works, audio uses the correct de
vice, or a paired phone responds.

4. Configuration and models

Create ~/.config/vladmir/config.json from config/config.example.json and re
strict its permissions:

```bash
mkdir -p ~/.config/vladmir
cp config/config.example.json ~/.config/vladmir/config.json
chmod 600 ~/.config/vladmir/config.json
```

Edit the JSON to use model identifiers present in Ollama. The example uses qwen3-vl:4b for chat and qwen2.5-coder:7b for programming, but those models must already be installed. Vladmir does not download models automatically. Check local models with ollama list.

Ollama uses loopback by default. keep_alive: 0 unloads a model after use to save VRAM, at the cost of loading it again on the next request. The first response may take longer. VLADMIR_CONFIG selects another configuration file ; other configuration variables use the VLADMIR_ prefix, as described in co nfig/README.md. Do not put tokens, passwords, or private URLs in Git.

Launch the UI with ./scripts/vladmir-desktop in the checkout or vladmir-des ktop after installing the launcher. ./scripts/vladmir opens voice/terminal mode according to the tool's options.

5. Conversations, memory, and modes

- General: primary chat with model selection and instant, medium, or high l evels. Levels adjust generation/context options; they do not guarantee bett er reasoning.
- Engineer: programming workspace with separate conversations and specific tools. Actual availability depends on the local model and enabled tools.
- Professor: study workspace with conversations and library material suppor t.
- Shared memory: context and facts may be shared among profiles through loc al storage. Memory does not retain every conversation forever. History sent to a model is limited by configuration and model context.
- Honcho: optional integration disabled by default. The example config sets share_conversations to false. Enabling a remote service may send data; con figure deliberately and review its privacy policy.

Persistent state is under ~/.local/state/vladmir and ~/.local/share/vladmir . Back these directories up safely before migration or cleanup. Exact locat ions may vary with XDG variables or configuration.

6. Attach files and use the library

Desktop chat accepts a set of text/structured documents, PDF, DOCX, and ima ges. Current chat limits are 5 attachments, up to 6 MiB per file, 8 MiB tot al, and 400,000 extracted text characters. The mobile API also limits reque st bodies and attachments. Therefore, 1 GB files are not supported in chat.

The Library is a separate workflow that imports and indexes references for later retrieval. Extractors cover documents, spreadsheets, presentations, e books, structured data, text, and many code formats. Relevant extractor lim its include 64 MiB input, 32 MiB expanded archive content, and 2 million ex tracted characters. Unsupported files may be represented by metadata or raw text; that does not mean their format was semantically understood.

To import, open Library, select files, and monitor the queue. Avoid importi ng huge directory copies, unknown executables, or unnecessary private data. Scanned PDFs require OCR; audio/video require local tools and extraction i s partial. The library does not automatically train the model on documents.

7. Voice, calls, and audio

The PC voice path captures mono audio at 16 kHz, detects speech, and uses W hisper for transcription. Spoken responses use Piper and a local fallback w hen available. The system uses configured PulseAudio/PipeWire input and out

put devices. App calls process audio in turns; they are not a telephone channel with guaranteed latency or end-to-end full-duplex speech.

How to use:

1. Install/configure input and output devices on the system.
2. Configure Whisper/Piper binaries and models as shown in config/config.example.json.
3. Choose Call or the voice tool and grant microphone access when requested by the OS.
4. If music or speech goes to the wrong device, check the desktop audio selector.
5. End the call to stop microphone capture.

Initial diagnosis: pactl list short sources, pactl list short sinks, pactl get-default-source, and pactl get-default-sink. A Bluetooth call may switch a headset to HFP/HSP, reducing playback quality. Using a separate input while keeping stereo A2DP output depends on local profiles and the audio server.

8. Camera and vision

On desktop, Vladmir can capture an image from a USB camera or a frame from a configured RTSP/RTSPS source and send it to a local vision model. Continuous vision limits frame rate and uses scene changes; it is not continuous recording or analysis of every frame. Select a source explicitly, check permissions, and test connectivity before relying on the feature.

RTSP URLs may contain a username/password: keep them only in local configuration with mode 0600. A URL check does not confirm that a physical camera or credentials work. Close the vision view to release capture.

9. Web, GitHub, and plugins

- Web: native plugin for search and reading public pages. Requires internet, an enabled integration, and an available provider. The reader restricts local/private destinations to reduce SSRF; it is not unrestricted access to any network.
- GitHub: can read public repositories or repositories accessible to the authenticated gh account. Editing is explicit in the panel, with branch selection and confirmation. This is not a generic remote shell.
- Plugins: the project loads reviewed native integrations (Web, GitHub, God's Eye View) and shows their permissions. An arbitrary third-party plugin marketplace is not implemented.
- JetBrains MCP: configure the local HTTP MCP server in PyCharm/CLion, enter its endpoint in Tools -> JetBrains IDEs, list the tools, and confirm each execution. Accept only tools and arguments you understand. Keep the server restricted to localhost.

For GitHub, install and authenticate gh according to its official documentation before opening the panel. Never paste a token into a conversation or repository. Actual access depends on account permissions and token scopes.

10. Agenda, email, packages, and Sentinel

- Agenda: items and reminders are stored in local SQLite. It can create, list, and complete simple events. It does not sync with Google Calendar or Outlook.
- Email: the tool prepares a mailto: draft and opens the configured application. The user reviews and sends it; Vladmir does not send directly through Tuta.
- Package installation: requests installation through allowed package managers, prepares a package inventory, and requires confirmation. The inventory is not a system image or full restore. AUR/PKGBUILD and unrestricted installation are restricted.
- Sentinel: UI for status, ClamAV scans of a selected path, and local inves

tigation. Install/update ClamAV and signatures yourself. It does not automatically scan the whole system with admin privileges. A clean result only applies to scanned items ClamAV could read. VirusTotal hash lookup is optional and disabled by default.

## 11. Self-evolution and training

The experimental engine gathers technical signals, creates proposals, and evaluates candidates in an isolated environment. Candidates are not automatically applied to production; promotion and rollback are protected actions. Scheduled runs depend on user systemd files being prepared and explicitly enabled.

Adapter/QLoRA training is a separate path. It requires an explicit authorization contract, reviewed dataset, hashes, and round/time/VRAM limits. Results are candidates evaluated against quality/efficiency; they do not prove general improvement or replace the model automatically. Training does not start if required artifacts, contract, or authorization are absent.

## 12. Android and Wear OS

Native source is under mobile-android/, vladmir-companion/, and vladmir-wear/. Use Android Studio with compatible SDK/JDK and consult the READMEs and Gradle wrappers. Install a debug build on a test device; release builds require signing configured outside Git.

The mobile app provides chat with a configured server, camera upload, turn-based voice calls, and optional ARCore. Connectivity requires the PC to be reachable, API running, valid TLS/address for remote use, pairing, and permissions. AR requires supported ARCore and Google Play Services for AR. Source indicates the AR map exists only while the screen is open; there is no automatic map persistence.

Companion forwards selected notifications and utilities over paired Bluetooth. WhatsApp is selectable; its text preview is limited and depends on Android notification access. Companion diagnostics say automatic reconnection is not yet enabled. Do not assume a permanent background link without validating the current device/version.

Listed sensors depend on watch hardware and Android permissions. The list indicates availability, not clinical calibration or accuracy. Heart rate, GPS, motion, notifications, and calls need individual tests on a physical device.

## 13. Privacy, backups, and security

- Keep ~/.config/vladmir/config.json at mode 0600 and out of version control.
- Back up configuration, memory, agenda, and library before upgrading.
- Keep the mobile API on loopback; use HTTPS/authenticated tunneling for remote access and do not expose an open port to the internet.
- Review enabled external integrations, especially Web, Honcho, GitHub, and optional hash lookup.
- Treat webpage, document, and repository content as untrusted data.
- See SECURITY.en.md for private vulnerability reporting and third-party license files for notices.

## 14. Troubleshooting checklist

1. Run ./scripts/vladmir --check and inspect missing dependencies.
2. Check ollama list and confirm the service responds at 127.0.0.1:11434.
3. Inspect Vladmir logs without copying tokens, private URLs, or transcripts into public support requests.
4. For audio, check Pulse sources/outputs and Bluetooth profile.
5. For camera, validate permissions and frame capture separately from model inference.

6. For phone/watch, check API, address, TLS, pairing, permissions, and background service.
7. For the library, check actual format, size, OCR/transcription dependencies, and item state.
8. If something fails, record version/commit, minimal steps, sanitized error, and reproducibility.

15. Internal references

Root README; config/config.example.json; src/mark/config.py; src/mark/desktop.py; src/mark/mobile_api.py; src/mark/attachment_limits.py; src/mark/cognition/library_extract.py; src/mark/audio.py; src/mark/tts.py; src/mark/vision.py; src/mark/plugins.py; src/mark/github.py; src/mark/jetbrains_mcp.py; src/mark/agenda.py; src/mark/package_install.py; src/mark/sentinel.py; src/mark/evolution/; mobile-android/README.md; vladmir-companion/; vladmir-wear/; SECURITY.en.md‘.